



FIRST SEMESTER, 2026–2027
Course Handout Part II

Date: 2nd August 2026

In addition to part I (General Handout for all courses appended to the Timetable), this portion gives further specific details regarding the course.

Course No : CS F301
Course Title : Principles of Programming Languages
Instructor-in-Charge : Amit Dua (amit.dua@pilani.bits-pilani.ac.in)
Instructors : **Lecture:** Amit Dua, and Mukesh Kumar Rohil (rohil@pilani.bits-pilani.ac.in)
Tutorial: Amit Dua, Rudresh Pillai (p20240029@pilani.bits-pilani.ac.in), and Aravind B S (p20220419@pilani.bits-pilani.ac.in), Akanksha Rao (p20250407@pilani.bits-pilani.ac.in), Degaga Wolde Feyisa (p20260804@pilani.bits-pilani.ac.in)

1. Course Description:

The course covers features of programming languages and introduces the main programming paradigms. It covers, in detail, the semantics of the features of programming languages – Control Abstraction, Data Types, and Data Abstraction, Scope and Parameter passing, inheritance, polymorphism and Concurrency related features. It covers various aspects of runtime environments like global and local data, code, function call stacks, dynamically allocated data, runtime features for exceptions and threads. Different programming paradigms, namely imperative, functional, object oriented and logic programming will be discussed in detail. Students will be introduced to the significance of a construct and how different languages implement that. The Lambda calculus will follow an introduction to functional programming languages. Formal elements of logic programming and programming tasks that explore the logic paradigm will be discussed.

2. Scope & Objective of the Course:

- Develop familiarity with the different paradigms of languages (imperative, functional, object oriented and logic) in programming and understanding the features of programming languages like abstraction, data types, etc.
- Understand the aspects of run-time environments like global and local data, function call stacks, and static and dynamic allocation of data.
- Introduction to the syntax and semantics of common functional programming languages.
- Students will be encouraged to learn the design aspects of languages belonging to different paradigms rather than learning a new language.
- Understand the basis of logic programming and basic elements of prolog.
- Understand synchronization features in concurrent programming.

3. Textbooks:

TB1. *Programming Languages: Concepts and Constructs*, Ravi Sethi, 2nd Edition, Addison Wesley, 1996.

References:

Ref1. *Concepts of Programming Languages*, Robert W. Sebesta, 10th Edition, Pearson Publishers, 2012.

Ref2. *An Introduction to Functional Programming Through Lambda Calculus*, Greg Michaelson, Dover Publication, New York, 2011.

Ref3. *Types and Programming Languages*, Benjamin C. Pierce, The MIT Press, 2002.

4. Course Plan:

Weekly plan for 17 teaching weeks (3 August to 27 November 2026) with 2 lectures per week

Week no.	Module	Lecture No.	Lecture details	Reference	Learning Outcomes
1.	Language Paradigms and Evaluation Criteria	L1	Introduction and Motivation, Language Paradigms, Imperative vs. Declarative Styles of Programming; Programming Languages that support these styles, Compilers. Language evaluation criteria.	TB1 Ch. 1	Identifying the language paradigms and language evaluation criteria
1.	Syntax and Language Description	L2	Lexical patterns in a program, language recognizers, formal methods of describing syntax, parse trees, attribute grammar, static and dynamic semantics.	TB1 Ch. 2; Ref1 Ch. 3	Understanding the formal methods of describing the syntax and the meaning of program constructs
2.	Names, Bindings, Scope and Lifetime	L3	Names, variables, bindings, scope and lifetime, referencing environment, static and dynamic type binding, storage binding and lifetime, categorization of static variables based on their lifetime.	Ref1 Ch. 5	Understanding of fundamental semantic issues of variables
2.	Control Abstraction	L4	Statements, control abstraction, Structured programming, syntax directed control flow, conditional statements, Loops and jumps, Case statements	TB1 Ch. 3; Ref1 Ch. 8	Understanding the need for Structured Programming and Syntax Directed Control Flow, programming with invariants
3.	Data Types and Type Systems	L5	Data Representation, Primitive and Structured Data types, ADTs; data type descriptor or type expression, enumerated data type, subrange data type, static layout decisions	TB1 Ch. 4; Ref1 Ch. 6	Identifying the Primitive Data Types, User defined ordinal types and constructed types. Understanding the concepts of type checking
3.	Data Types and Type Systems	L6	Array types, design issues, indices, array bounds, subscript binding and array categories, array layout, relative addresses and efficient address computations, array initializations and operations, jagged and associative arrays, static and dynamic array bounds.	TB1 Ch. 4; Ref1 Ch. 6	Understanding of the design issues for array data type
4.	Data Types and Type Systems	L7	Records Structures, Variants and union, component selection, operations on records, static layout, design issues, layout of variant records.	TB1 Ch. 4; Ref1 Ch. 6	Understanding of the design issues for record data type
5.	Data Types and Type Systems	L8	Sets, pointers, dynamic structures. Type checking vs. Type Inferencing, Type Equivalence and Subtyping.	TB1 Ch. 4; Ref1 Ch. 6	Understanding of the design issues for sets and pointers data types and significance of type checking
5.–6.	Subprograms and the Run-Time Environment	L9–L10	Introduction to procedures, Recursion, Parameter Passing Methods, Call-by-Value, Call-by-Reference, Call by Value Result. Scope rules for names, static and dynamic scope rules, nested scope, Activation Records, Lexical Scope, Dangling Pointers	TB1 Ch. 5; Ref1 Ch. 5, 9, 10	Understanding the run time environment, semantics of calls and returns, implementing subprograms, implementing static scoping and dynamic scoping

Week no.	Module	Lecture No.	Lecture details	Reference	Learning Outcomes
6.–8.	Functional Programming	L11–L13	Introduction and basic elements of Functional Programming, Lists and associated Operations, Function Declaration, Predicate Functions, Tail recursive functions, Higher-Order Functions, Polymorphism	TB1 Ch. 9, 10; Ref2 Ch. 1, 2	Understanding the basics of functional programming and the benefits of functional programming
8.–11.	Lambda Calculus	L14–L16	Combinators	Ref2 Ch. 5, 6	Understanding lambda calculus, and using type systems as a way to reason about programs. Combinators necessary for any problem solving and representation
11.	Lambda Calculus	L17–L18	Composition and evaluation	Ref2 Ch. 7, 8	Understanding and practicing the evaluation including Composition
12.–13.	Object-Oriented Programming	L19–L21	Object-Oriented Programming Paradigm and features, Class hierarchy, Inheritance, Information hiding.	TB1 Ch. 7; Ref1 Ch. 12	Understanding the OOP concepts and design issues, implementing dynamic binding and inheritance in OOP languages
14.–16.	Logic and Concurrent Programming	L22–L26	Different programming paradigms: Logic, concurrent, parallel and Visual programming.	Class Notes	Understand synchronization features in concurrent, Logic and Visual programming

Tutorial Plan:

Week no.	Tutorial No.	Topic
2 (10 – 14 Aug)	T1	Language paradigms and evaluation criteria
3 (17 – 21 Aug)	T2	Syntax description
4 (24 – 28 Aug)	T3	Names, binding, scope and lifetime
5 (31 Aug – 4 Sep)	T4	Control abstraction
6 (7 – 11 Sep)	T5	Primitive and structured data types
7 (14 – 18 Sep)	T6	Array types
8 (21 – 25 Sep)	T7	Records, unions and variants
9 (28 Sep – 2 Oct)	T8	Pointers and type systems
11 (12 – 16 Oct)	T9	Subprograms and parameter passing
13 (26 – 30 Oct)	T10	Functional programming
14 (2 – 6 Nov)	T11	Lambda calculus: reduction
15 (9 – 13 Nov)	T12	Combinators and evaluation order
16 (16 – 20 Nov)	T13	Objects, logic and concurrency

5. Evaluation Scheme:

Component	Wt (%)	Duration	Date & Time	Comments
Quiz 1	12.5	35–40 mins.	2nd September 2026	Closed Book

Component	Wt (%)	Duration	Date & Time	Comments
Quiz 2	12.5	35–40 mins.	30th September 2026	Closed Book
Mid-Semester Test	30	90 mins.	8th October 2026 (FN)	Open Book
Comprehensive Examination	45	180 mins.	1st December 2026 (FN)	Closed Book

Exact venue for Quizzes will be uploaded on Nalanda.

Instructions for the open book component (Mid-Semester Test):

- **Permitted:** printed textbooks and reference books, and handwritten notes written in a bound notebook.
- **Not permitted:** photocopied material from any book, loose or unbound handwritten sheets, printouts of slides, printed or photocopied lecture notes, and any other printed or reproduced material.
- Material may not be shared or exchanged with another student during the test.

NC Criterion:

- Any student who fails to write the mid-semester examination or the comprehensive examination (regular or make-up) will be awarded NC.
- Outliers are identified statistically. A student whose total marks fall below (mean – 3 × standard deviation) of the class total will be awarded NC (If distribution is not skewed).

6. Office Consultation Hour:

Amit Dua (Monday, 10:00 – 10:50 AM). Mukesh Kumar Rohil (Monday 11:00-11:50 AM). You may also email to seek a prior appointment.

7. Notices:

All notices will be posted on the course LMS hosted at <https://nalanda.bits-pilani.ac.in/>

8. General Instructions:

- Mobile phones must be kept inside your bag during lecture and tutorial hours.
- Do not use headphones or earphones in lectures and tutorials.
- Mobile phones are to be used only for marking attendance and for class participation activities conducted by the instructor.

9. Make-up Policy:

Make-up will be given in case of extreme medical emergency or hospitalization only.

Instructor-in-charge
CS F301 Principles of Programming Languages